AUTOMATION TRACK · CAPSTONE

Fleet Fuel Watchdog

Build a no-code automation that checks every fuel purchase as it comes in, emails the manager when something looks wrong, and sends a weekly report by itself.

DATASET
Real fleet fuel records: 349 past fills + 26 new test entries

TOOLS
Google Sheets, Google Forms, Gmail and n8n (or a free alternative)

DELIVERABLE
Two working workflows, your Google Sheet and a 3–5 min demo video

Student guide and submission pack · Rotech Data Consult

SECTION 1

The Brief

You've just joined the Fleet Unit of a railway construction company as its Automation Assistant. The Unit runs Toyota Hilux pickups on two project sites, KMD and RKK.

Every time a driver buys fuel, someone writes it in a log. Nobody checks those entries until the end of the month, by which time the money is gone. In September 2024 the pump price jumped from about ₦950 to over ₦1,100 a litre, so every wasted litre now hurts. The Fleet Manager's message to you:

> **MESSAGE FROM THE FLEET MANAGER**
> "I don't want to find problems a month late. When a driver logs fuel, I want it checked straight away: is the price right, is the vehicle doing the kilometres it should, did someone type nonsense? If something's wrong, tell me by email that same day. And every Monday, send me a short summary so I don't have to open the spreadsheet."

What you'll build

Workflow 1 · Fuel Checker

Runs every time a new fuel entry arrives. It calculates price per litre and km per litre, applies the Unit's rules, saves the result and emails an alert if anything is wrong.

Workflow 2 · Weekly Report

Runs on a schedule. It reads the week's entries, totals spend, litres and efficiency per project, and emails the manager a tidy summary.

How the pieces connect

1. Driver	2. Sheet	3. Check	4. Record	5. Alert
fills a Google Form	response row lands	calculate + rules	Fuel_Log + status	email if flagged

...and separately, every Monday: Schedule → read the week's rows → summarise → email the report.

What you'll practise

Triggers

- "New row added" (polling) trigger
- Schedule trigger + a manual trigger for testing

Working with data

- Reading, looking up, appending and updating Google Sheet rows
- Expressions / formulas inside a workflow

Logic

- IF / Router / Switch branches
- Reading settings from a sheet instead of hard-coding them

Communication

- Sending emails with data filled in
- Logging what the automation did, for an audit trail

SECTION 2

Choose Your Tool

n8n Cloud's free trial only lasts 14 days, so yours may have expired. That's fine: you don't need to pay for anything in this project. Choose **one** of the routes below.

RECOMMENDED

Route A · n8n on your laptop

- **Free, no limit** on runs or workflows
- Same tool and same nodes you learned in class
- Needs Node.js (or Docker) installed. Setup in **Appendix A**
- Only runs while your laptop and n8n are on

NO INSTALL

Route B · Make.com

- Free plan in the browser, connects to Google easily
- Monthly cap on operations and at most 2 active scenarios. Test with "Run once" so you don't use it up
- Node mapping in **Appendix B**

NO INSTALL

Route C · Activepieces

- Open-source and looks a lot like n8n
- Free cloud plan with a monthly task cap
- Fewer tutorials, so choose it only if you like exploring

BEFORE YOU CHOOSE

Free-plan limits change often. Check the pricing page of Make or Activepieces on the day you sign up. If your n8n Cloud trial is **still active**, you may use it, but export your workflows regularly (*Download* → JSON) so you don't lose them when the trial ends.

Why not Zapier?

Its free plan only allows two-step automations (one trigger, one action). Our Fuel Checker needs about eight steps, so it won't fit.

Same ideas, different names

What you need	n8n	Make.com	Activepieces
Start on new row	Google Sheets Trigger → *Row Added*	Google Sheets → *Watch New Rows*	Google Sheets → *New Row Added*
Start on a timetable	Schedule Trigger	Scenario schedule (clock icon)	Schedule → *Every Week*
Look up a row	Google Sheets → *Get Row(s)* + filter	Google Sheets → *Search Rows*	Google Sheets → *Find Rows*
Calculate values	Edit Fields (Set) or Code	Set multiple variables / inline formulas	Code step or inline formulas
Branch on a condition	IF / Switch	Router + filters	Branch / Router
Add a row	Google Sheets → *Append Row*	Google Sheets → *Add a Row*	Google Sheets → *Insert Row*
Send email	Gmail → *Send* (or Send Email via SMTP)	Gmail → *Send an Email* (or Email via SMTP)	Gmail → *Send Email* (or SMTP)
Group & total	Summarize node, or Code	Numeric / Table aggregator	Code step or loop

The rest of this guide uses n8n names. If you're on another route, use this table to find the matching step.

SECTION 3

Your Files and Your Sheet

Files you'll receive

File	What it is
`Fleet_Watchdog_Starter.xlsx`	The starting database. You'll upload it to Google Drive and open it as a Google Sheet. It has 5 tabs, listed below.
`New_Fuel_Entries_Test_Week.csv`	26 fuel purchases from 2–8 September 2024. These are your **test data**: you feed them in once your workflow is built. The columns match the Google Form exactly. **Don't open it and fix anything.** Some rows are wrong on purpose.

The tabs in the starter sheet

Tab	Columns	Purpose
`Fuel_Log`	Entry_ID, Date, Project, Vehicle, Driver, Odometer_km, Distance_km, Litres, Amount_NGN, Price_per_Litre, Km_per_Litre, Status, Flags, Processed_At	The master record. Already holds 349 past fills (June–Aug 2024, Status = `HISTORICAL`). Your workflow **appends** each new checked entry here.
`Vehicles`	Vehicle, Project, Make_Model, Expected_Kml, Tank_Capacity_L	8 vehicles. *Expected_Kml* is the Fleet Unit's target km per litre for each one.
`Settings`	Setting, Value	Alert email, price band, efficiency threshold and report dates. **Your workflow must read these values, not type them in.**
`Alerts_Log`	Alert_ID, Sent_At, Entry_ID, Vehicle, Driver, Flags, Message	Empty. Add one row for every alert email you send.
`Weekly_Reports`	Sent_At, Period_Start, Period_End, Fills, Spend_NGN, Litres, Km_per_Litre, Flagged	Empty. Add one row each time the weekly report runs.

The Google Form (you build it in Task 2)

Name it **Fleet Fuel Entry**. Add these questions **in this order**, using these exact titles:

#	Question title	Type	Notes
1	Date	Date	Date of the fuel purchase
2	Project	Dropdown	KMD, RKK
3	Vehicle	Dropdown	The 8 plates from the Vehicles tab
4	Driver	Short answer	Leave it **not required**. We want to see what happens when it's blank
5	Odometer_km	Short answer (number)	Reading on the dashboard
6	Distance_km	Short answer (number)	Km driven since the last fill
7	Litres	Short answer (number)	Litres bought
8	Amount_NGN	Short answer (number)	Naira paid

Link the form to **your** Watchdog sheet (*Responses → Link to Sheets → Select existing spreadsheet*). Google adds a new tab, usually called `Form Responses 1`, with a *Timestamp* column first. Your trigger watches that tab.

SECTION 4

The Fleet Unit's Rules

Your Fuel Checker applies these rules to every new entry. A single entry can break more than one rule. List every rule it breaks.

Step 1: calculate

Field	Formula	Round to
Price_per_Litre	`Amount_NGN ÷ Litres`	2 decimal places
Km_per_Litre	`Distance_km ÷ Litres`	2 decimal places

Step 2: check

Flag	Raise it when...	Values come from
`INCOMPLETE`	Vehicle, Driver, Odometer_km, Distance_km, Litres or Amount_NGN is blank. **Stop here** and don't calculate anything.	The form response
`PRICE`	Price_per_Litre is **below** Min_Price_per_Litre **or above** Max_Price_per_Litre	Settings tab
`LOW_EFFICIENCY`	Km_per_Litre < Expected_Kml × Efficiency_Threshold	Vehicles tab (Expected_Kml) + Settings tab
`OVERFILL`	Litres > Tank_Capacity_L	Vehicles tab

Step 3: set the status

Status	When
`INCOMPLETE`	The INCOMPLETE flag was raised
`FLAGGED`	Any other flag was raised. Put all of them in *Flags*, comma-separated, e.g. `PRICE, OVERFILL`
`OK`	No flags. Leave *Flags* empty

Both `FLAGGED` and `INCOMPLETE` entries trigger an alert email. `OK` entries don't.

WORKED EXAMPLE: ROW 1 OF THE TEST FILE

19A 111 FG, 2 Sept 2024. Distance 372 km, Litres 52.63, Amount ₦50,000.
Price_per_Litre = 50,000 ÷ 52.63 = **950.03**. That's inside the band of ₦900–₦1,250, so no PRICE flag.
Km_per_Litre = 372 ÷ 52.63 = **7.07**. The minimum allowed is 6.8 × 0.75 = 5.10, so no LOW_EFFICIENCY flag.
52.63 L is under the 80 L tank, so no OVERFILL flag. → Status **OK**, Flags empty, no email.

WHY THE RULES LIVE IN THE SETTINGS TAB

Fuel prices change every few weeks. When they do, the manager should only have to edit one cell in Settings, not open your workflow. A workflow with ₦900 or 0.75 typed directly into a node loses marks in Task 4.

Bonus rule (optional)

Flag	Raise it when...
`ODOMETER`	The new Odometer_km is more than Odometer_Tolerance_km away from (the vehicle's **last** odometer in Fuel_Log + Distance_km). It catches mistyped readings and fills that someone forgot to log.

SECTION 5

Your Tasks

1 Set up your tool and your sheet

10 marks

- [ ] Choose your route (Section 2) and get it running. For Route A, follow Appendix A.
- [ ] Upload `Fleet_Watchdog_Starter.xlsx` to Google Drive. Open it with Google Sheets and use *File → Save as Google Sheets*. Rename it **Fleet Watchdog – YourName**.
- [ ] In the Settings tab, change *Alert_Email* to your own email address.
- [ ] Connect your tool to your Google account (Sheets and Gmail). Test it with a tiny workflow that reads one row from Vehicles.

2 Build the intake form

10 marks

- [ ] Build the **Fleet Fuel Entry** form exactly as shown in Section 3: same titles, same order.
- [ ] Link it to your Watchdog sheet. Check that the `Form Responses 1` tab appears.
- [ ] Submit one test entry yourself and check it appears in the tab. Then **delete that test row** before moving on.

3 Workflow 1, part A: catch and calculate

15 marks

Create a workflow called **Fuel Checker**.

- [ ] **Trigger:** Google Sheets Trigger → *Row Added* on `Form Responses 1`.
- [ ] **Look up the vehicle:** Google Sheets → *Get Row(s)* from Vehicles, filtered on the Vehicle from the form, to get Expected_Kml and Tank_Capacity_L.
- [ ] **Read settings:** Google Sheets → *Get Row(s)* from Settings. Tip: in n8n you can turn the rows into one object with a small Code node, or reference each row with an expression.
- [ ] **Calculate:** use an Edit Fields (Set) node to create Price_per_Litre and Km_per_Litre, rounded to 2 decimals.
- [ ] Make an **Entry_ID** for the entry, e.g. `W-` + the form timestamp with the symbols removed, so each one is unique.

Suggested shape of Workflow 1 (Tasks 3–5)

Sheets Trigger → Get Vehicle → Get Settings → Calculate → Apply Rules → Append to Fuel_Log →
IF Status ≠ OK → Gmail Alert → Append Alerts_Log

TEST AS YOU GO

Add one row to `Form Responses 1`, then in n8n click *Fetch Test Event* on the trigger and run each node with *Execute step*. Check the output panel before you add the next node. Building all nine nodes and only then pressing Run is the most common way students get stuck.

4 Workflow 1, part B: apply the rules and save

20 marks

- [ ] Apply all four rules from Section 4. Two ways that both earn full marks: (a) one Code node that builds the Flags list, or (b) IF nodes plus a Set node that joins the results. Choose the one you can explain.
- [ ] Set Status to `OK`, `FLAGGED` or `INCOMPLETE`.
- [ ] Every threshold comes from the Settings or Vehicles tab. **Nothing is typed into the node.**
- [ ] **Append** the checked entry to Fuel_Log with every column filled in, and Processed_At set to the current date and time.

5 Workflow 1, part C: send the alert

15 marks

- [ ] An IF node lets only `FLAGGED` / `INCOMPLETE` entries through.
- [ ] A Gmail node sends to *Alert_Email* from Settings. The subject line should say what happened at a glance, e.g. ⚠ `Fuel alert – 19A 240 FG – LOW_EFFICIENCY`.
- [ ] The email body includes: date, vehicle, driver, litres, amount, price per litre, km per litre, the vehicle's expected km/l and a one-line plain-English explanation of each flag.
- [ ] After sending, append a row to **Alerts_Log**.
- [ ] Turn the workflow **on** (Active / Published) so it runs without you clicking anything.

6 Workflow 2: the weekly report

15 marks

Create a second workflow called **Weekly Report**.

- [ ] **Triggers:** a Schedule Trigger for every Monday at 08:00, **plus** a Manual Trigger so you can test it now.
- [ ] Read *Report_Start_Date* and *Report_End_Date* from Settings. Get Fuel_Log rows whose Date falls between them (inclusive) and whose Status isn't `INCOMPLETE`.
- [ ] For each project **and** overall, work out: number of fills, total litres, total spend (₦), total distance, km per litre (total distance ÷ total litres) and the number of FLAGGED entries.
- [ ] Email a clean HTML report with a small table, the top 3 vehicles with the most flags, and one sentence on how efficiency changed from the previous period, if you can.
- [ ] Append one row to **Weekly_Reports**.

> **WHY THE DATES COME FROM SETTINGS**
>
> Our data is from 2024, so "the last 7 days" from today would find nothing. In a real deployment you'd swap the Settings dates for an expression like `{{ $now.minus({days: 7}) }}`. Say in your demo video how you would do that.

7 The test week: run it for real

10 marks

With the Fuel Checker **active**, feed in all 26 rows from `New_Fuel_Entries_Test_Week.csv`. Choose one of these methods:

- **Method 1 (realistic):** submit 3–4 rows through your Google Form by hand.
- **Method 2 (fast):** copy rows from the CSV and paste them under the last row of `Form Responses 1`. Paste **5 rows at a time** and wait for the workflow to pick them up before pasting more.

Using both methods is fine, as long as every row goes in exactly once. Then run Workflow 2 with the Manual Trigger.

- [ ] Fuel_Log now has 349 + 26 = **375** rows.
- [ ] Every new row has a Status. Every FLAGGED / INCOMPLETE row has a matching email **and** an Alerts_Log row.
- [ ] Weekly_Reports has a row for 2–8 Sept 2024, and the report email arrived.
- [ ] Take screenshots of: the Fuel_Log new rows, Alerts_Log, one alert email, the weekly report email and both workflow canvases.

SENSE-CHECK YOUR RESULTS

Your instructor has the answer key. Before you submit, check these yourself:

- Row 1 (19A 111 FG, 2 Sept) should be **OK**, with 950.03 ₦/L and 7.07 km/L.
- Prices that week sit between ₦950 and ₦1,300. Only a few entries should break the price band. If half your entries say PRICE, look at your Settings lookup.
- At least one row has a blank driver, and at least one has a number that's clearly mistyped. Did your workflow catch them?
- Look at the weekly km per litre for each project. Does one entry distort it? Mention this in your video.

Common problems

Symptom	Likely cause / fix
Trigger never fires	Workflow isn't active, n8n isn't running (Route A), or you're watching the wrong tab. The Sheets trigger checks every minute or so, so give it time.
Numbers come out as text ("52.63")	Form answers arrive as text. Convert them, e.g. `Number($json.Litres)` or `parseFloat()`.
Vehicle lookup returns nothing	Extra spaces or different spelling. The dropdown values must match the Vehicles tab exactly.
One entry, two emails	Your IF node passes items down both branches, or the trigger read the row twice. Check the execution log.
Weekly report finds 0 rows	Comparing a date to a text string. Turn both into dates before comparing.
"Access blocked" when connecting Google	Route A: add yourself as a *Test user* on the OAuth consent screen (Appendix A, step 5).

SECTION 6

Submission and Marking

What to submit

#	Item	Details
1	Workflow files	n8n: ⋯ → *Download* for each workflow (two `.json` files). Make: *Export Blueprint*. Activepieces: *Export flow*. **Remove passwords and API keys** before sharing.
2	Google Sheet link	Shared as *Anyone with the link → Viewer*.
3	Demo video (3–5 min)	Screen recording: submit one new entry live and show the alert arriving, walk through both workflows, run the weekly report, and explain one design choice. Loom free or any screen recorder.
4	One-page README	Which route you chose and why, a diagram or screenshot of each workflow, your test-week results (how many OK / FLAGGED / INCOMPLETE) and two improvements you'd make next.

Marking scheme

Task	What we look for	Marks
1	Tool running, sheet set up, Google connected	10
2	Form built to spec and linked to the sheet	10
3	Trigger, both lookups and correct calculations	15
4	All four rules right, status right, thresholds read from sheets, Fuel_Log appended	20
5	Alert only when needed, informative email, Alerts_Log filled	15
6	Scheduled report with correct totals per project, emailed and logged	15
7	All 26 test entries processed; results match the answer key	10
–	Demo video and README: clear, honest, explains choices	5
	Total	**100**

Bonus (up to +10)

Challenge	Marks
ODOMETER rule from Section 4 (look up the vehicle's last Fuel_Log row)	+4
A second alert channel: Telegram bot or WhatsApp (e.g. via a free Telegram bot token)	+3
An error workflow that emails you if the Fuel Checker crashes	+3

DEADLINES AND WHERE TO SUBMIT

Deadline: _____ Submit to: _____

ACADEMIC HONESTY

You may use AI assistants, YouTube and the n8n community forum. Your video must show **you** explaining your own workflow. If you can't explain a node, you'll lose the marks for it.

APPENDIX A

Running n8n Free on Your Laptop

This is n8n's free self-hosted Community Edition. Same canvas, same nodes, no time limit.

Option 1: with Node.js (simplest)

1. Install **Node.js LTS** (version 20 or 22) from `nodejs.org`. Accept the defaults.
2. Open **Command Prompt** (Windows) or **Terminal** (Mac) and run:

```
npx n8n
```

3. Type `y` if asked to install. The first start takes a few minutes.
4. When you see *Editor is now accessible via http://localhost:5678*, open that address in your browser and create your local owner account.
5. Keep that window open. **If you close it, n8n stops** and your triggers stop too. Next time, just run `npx n8n` again. Your workflows are saved.

Option 2: with Docker Desktop

```
docker volume create n8n_data
docker run -it --rm --name n8n -p 5678:5678 -v n8n_data:/home/node/.n8n docker.n8n.io/n8nio/n8n
```

Connecting Google (one-time, about 15 minutes)

n8n Cloud does this for you. On your own laptop you create your own free "Google app" so n8n is allowed to use your Sheets and Gmail.

1. Go to `console.cloud.google.com` and create a project called **n8n-local**.
2. *APIs & Services* → *Library*: enable **Google Sheets API**, **Google Drive API** and **Gmail API**.
3. In n8n, create a credential *Google Sheets OAuth2 API* and copy the **OAuth Redirect URL** it shows (it looks like `http://localhost:5678/rest/oauth2-credential/callback`).
4. Back in Google: *OAuth consent screen* → User type **External** → fill in the app name and your email.
5. Under **Test users** (in the consent screen / Audience settings), add **your own Gmail address**. Skipping this causes the "Access blocked" error.
6. *Credentials* → *Create credentials* → *OAuth client ID* → type **Web application** → under *Authorised redirect URIs* paste the URL from step 3 → Create.
7. Copy the **Client ID** and **Client Secret** into the n8n credential and click **Sign in with Google**. If Google warns the app is unverified, click *Continue*. It's your own app.
8. Do the same for a *Gmail OAuth2* credential, reusing the same Client ID and Secret.

> **EASIER EMAIL OPTION**
>
> If Gmail OAuth gives you trouble, use n8n's **Send Email** node with Gmail SMTP: host `smtp.gmail.com`, port `465`, SSL on, user = your Gmail, password = a Google **App Password**. To get one, turn on 2-Step Verification, then go to *Google Account* → *Security* → *App passwords*. This also works on Make and Activepieces.

Screens in Google Cloud and n8n change from time to time. If a button has moved, search the n8n docs for "Google OAuth2 credentials".

APPENDIX B

Tips for the Make.com and Activepieces Routes

Make.com

- Sign up at `make.com` for the free plan. One **scenario** = one workflow, so you need two: *Fuel Checker* and *Weekly Report*. The free plan allows that.
- Fuel Checker starts with **Google Sheets** → **Watch New Rows** on `Form Responses 1`. When asked where to start, choose *From now on*.
- Use **Search Rows** for the Vehicles and Settings lookups. Do the maths with Make's inline functions, e.g. `round(Amount / Litres; 2)`. Use the `if()` function to build each flag.
- Use a **Router** with a filter (*Status not equal to OK*) before the email route.
- Weekly Report: **Search Rows** on Fuel_Log → **Numeric Aggregator** (Group by: Project) for the totals → **Text Aggregator** to build HTML table rows → Gmail.
- **Save your operations.** While building, use **Run once**, not the schedule. Set the Fuel Checker schedule to 15 minutes only for Task 7, and switch it off when you finish.

Activepieces

- Sign up at `activepieces.com` (free cloud plan). Flows look a lot like n8n.
- Trigger: **Google Sheets** → **New Row Added**. Lookups: **Find Rows**. Maths and rules: a **Code** step (JavaScript, almost the same as n8n's Code node).
- Branching: **Branch** step on Status. Email: **Gmail** → **Send Email** or the **SMTP** piece with an App Password.
- Publish the flow so it runs by itself. Watch your monthly task count in *Settings* → *Billing*.

A starting point for the rules Code step (n8n or Activepieces)

```
// inputs: entry (form row), veh (Vehicles row), cfg (Settings as {Setting: Value})
const flags = [];
const need = ['Vehicle','Driver','Odometer_km','Distance_km','Litres','Amount_NGN'];
if (need.some(k => entry[k] === '' || entry[k] == null)) {
  return { Status: 'INCOMPLETE', Flags: 'INCOMPLETE' };
}
const litres = Number(entry.Litres);
const price  = +(Number(entry.Amount_NGN) / litres).toFixed(2);
const kml    = +(Number(entry.Distance_km) / litres).toFixed(2);
if (price < Number(cfg.Min_Price_per_Litre) || price > Number(cfg.Max_Price_per_Litre))
flags.push('PRICE');
// TODO: LOW_EFFICIENCY rule
// TODO: OVERFILL rule
return { Price_per_Litre: price, Km_per_Litre: kml,
         Status: flags.length ? 'FLAGGED' : 'OK', Flags: flags.join(', ') };
```

In n8n's Code node you'll wrap this to read from `$input` / earlier nodes and return `[{ json: {...} }]`. Finishing the two TODO rules is part of Task 4.

FINISHED EARLY?

Point the Fuel Checker at a fresh copy of the form and ask a classmate to be your "driver" for a day. Real automations are judged on whether they keep working when someone else uses them.